CS 584 Natural Language Processing

Lecture 7: Pre-training and Post-training

Yue Ning
Department of Computer Science
Stevens Institute of Technology

Learning Objectives

- Be able to explain pre-training and post-training including algorithms and architectures

Two stages of language model training

- Pre-train
  - Train the LM on massive amount of data.
  - Goal: let the model learn generic language knowledge and world knowledge.
    - The scale and quality matter
    - Model architectures
- Post-train/Fine-tuning
  - Train the LM on task-specific data.
  - Goal: let the model have strong task performance.

The Pre-train / Post-train Paradigm

Step 1: Pretrain

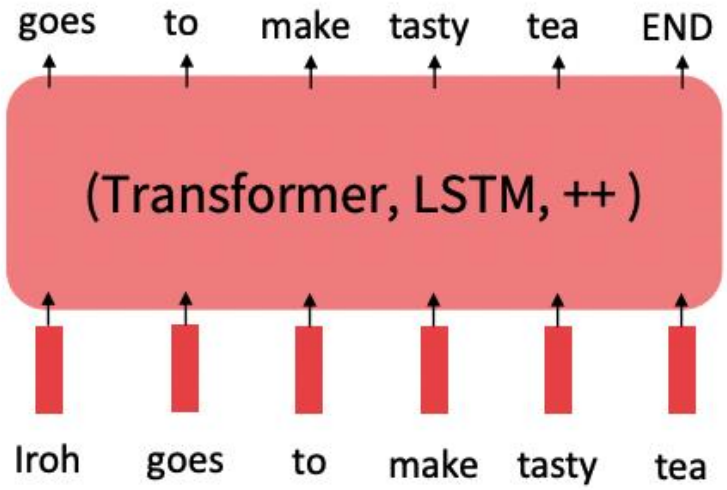
Popular algorithms:

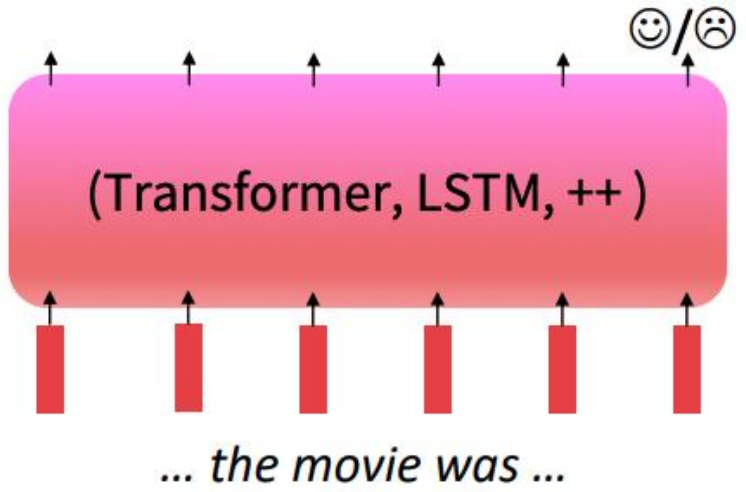
- Predict masked words
- Predict next words

Step 2: Post-train

Popular algorithms:

- Supervised fine-tuning
- Parameter-efficient fine-tuning
- Preference learning





Part 1

Pre-training

Why pretraining?

- On many tasks, common capabilities are needed, including:
  - Understand language's meaning
  - Commonsense
  - Language generation
  - etc.
- These capabilities require massive data to learn.
  - Nowadays, pretraining involves 99.9%+ of training data (in terms of number of tokens)
- With a model pre-trained, models do not need to learn these capabilities from scratch.

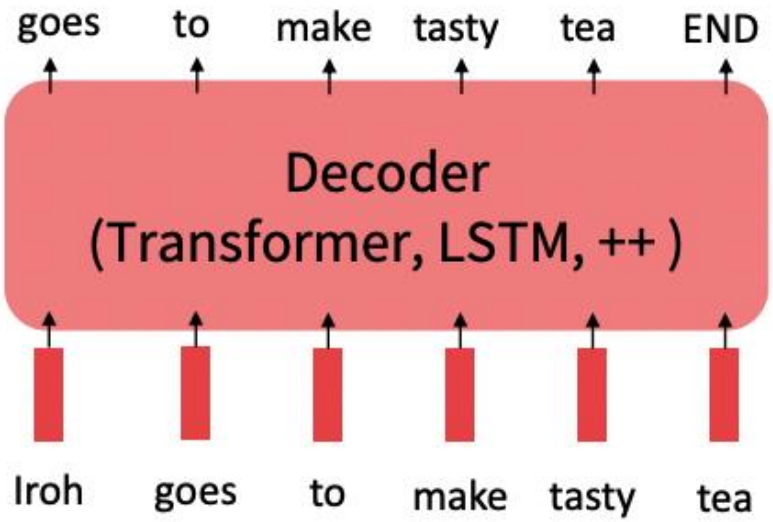
Pretraining through language modeling

Recall the language modeling task:

- Model $p_\theta(w_t|w_{1:t-1})$, the probability distribution over words given their past contexts.

Pretraining through language modeling:

- Train a neural network to perform language modeling on a large amount of text.
- Save the network parameters



[Dai and Le, 2015]

What can we learn from reconstructing the input?

One useful pretraining setting is to predict the masked words.

- I put ___ fork down on the table.
- The woman walked across the street, checking for traffic over ___ shoulder.
- I went to the ____ to see the fish, turtles, and seals.
- I was thinking about the sequence that goes 1, 1, 2, 3, ___, 8, 13, 21, ….

Models like **BERT** are pretrained with masked-word prediction (among other tasks)

What can we learn from continuing the sentence?

Another useful pretraining setting is to predict the next word.

- I put a fork down on the ____
- The woman walked across the street, checking for traffic over her ____
- I went to the ocean to see the fish, turtles, and ____
- I was thinking about the sequence that goes 1, 1, 2, 3, 5, 8, 13, 21, ____

Models like **GPT** are pretrained with next-word prediction (among other tasks)

Pretraining: Data collection

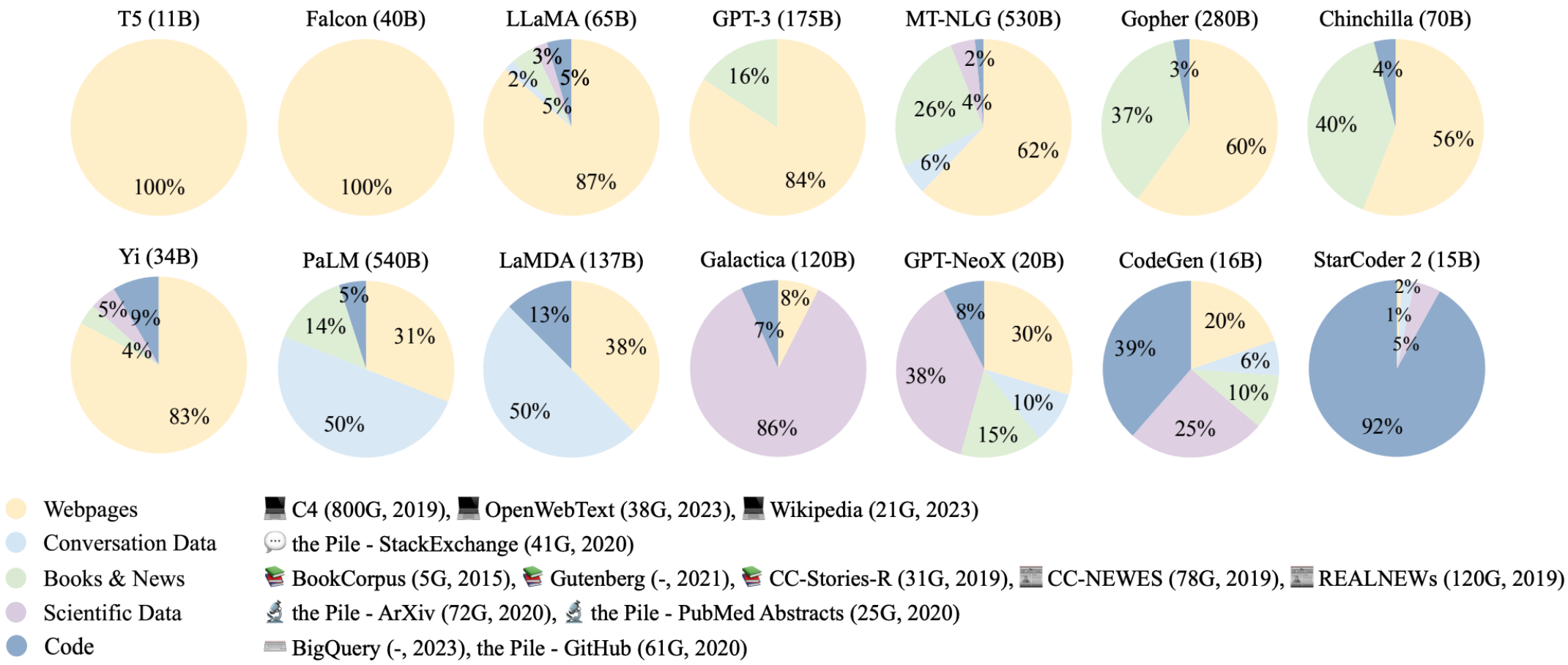


Fig. 6: Ratios of various data sources in the pre-training data for existing LLMs.

Figure from "A Survey of Large Language Models", Zhao et al (2023)

Pretraining: Data Source (two types)

- **(1) General data:** with large, diverse, and accessible nature to enhance the language modeling and generalization abilities of LMs.
  - Webpages
  - Books
  - Conversational text

Pretraining: Data Source (two types)

- **(2) Specialized data**: endowing LLMs with specific task-solving capabilities.
  - Multilingual data
  - Scientific data
  - Code

Pretraining: Data Preprocessing

Removing noisy, redundant, irrelevant, and potentially toxic data

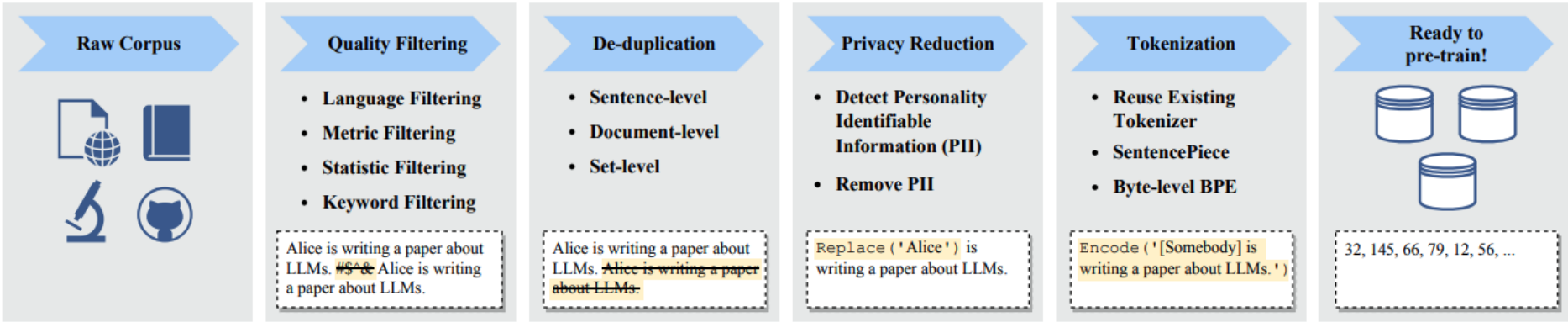


Fig. 6: An illustration of a typical data preprocessing pipeline for pre-training large language models.

Pretraining: Data Preprocessing

- Quality Filtering
  - Classifier-based and heuristic-based
- De-duplication
  - Granularities: sentence-level, document-level, and dataset-level
- Privacy Reduction
  - Remove the personally identifiable information (PII)
- Tokenization
  - Byte-Pair Encoding (BPE); WordPiece tokenization
  - Recent LLMs often train the customized tokenizers specially for the pre-training corpus.

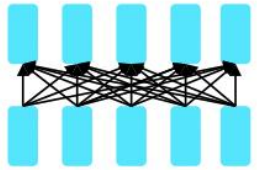
Effect of Pre-training Data on LLMs

- Infeasible to iterate the pre-training of LLMs multiple times
- Important: construct a well-prepared pre-training corpus.
- The quality and distribution of the pre-training corpus influence LLMs.
  - **Mixture of sources**
  - **Amount of Pre-training Data**
  - **Quality of Pre-training Data**

Pre-training Architectures

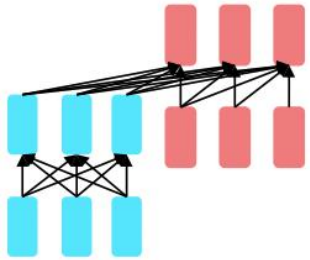
Three types of architectures

The neural architecture influences the type of pretraining, and natural use cases.



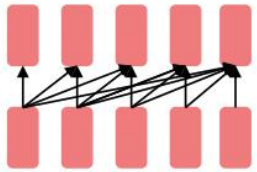
Encoders

- Gets bidirectional context – can condition on future!
- How do we train them to build strong representations?



Encoder-Decoders

- Good parts of decoders and encoders?
- What's the best way to pretrain them?



Decoders

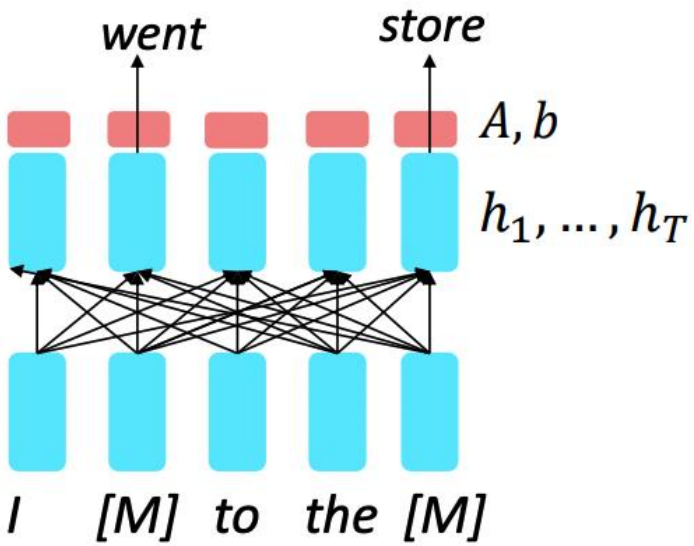
- Language models! What we've seen so far.
- Nice to generate from; can't condition on future words

Pretraining encoders

- Idea: replace some fraction of words in the input with a special [MASK] token; predict these words.

$$h_1, \ldots, h_T = \text{Encoder}(w_1, \ldots, w_T)$$
$$y_i \sim Aw_i + b$$

- Only add loss terms from words that are "masked out." If $\tilde{x}$ is the masked version of x, we're learning $p_\theta(x|\tilde{x})$. Called Masked LM.

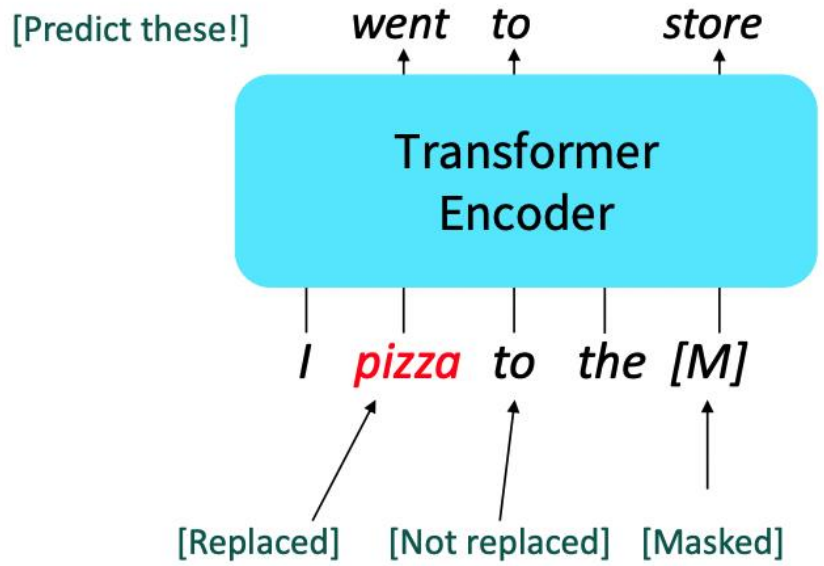


[Devlin et al., 2018]

BERT: Bidirectional Encoder Representations from Transformers

Pre-training task (1): Masked LM

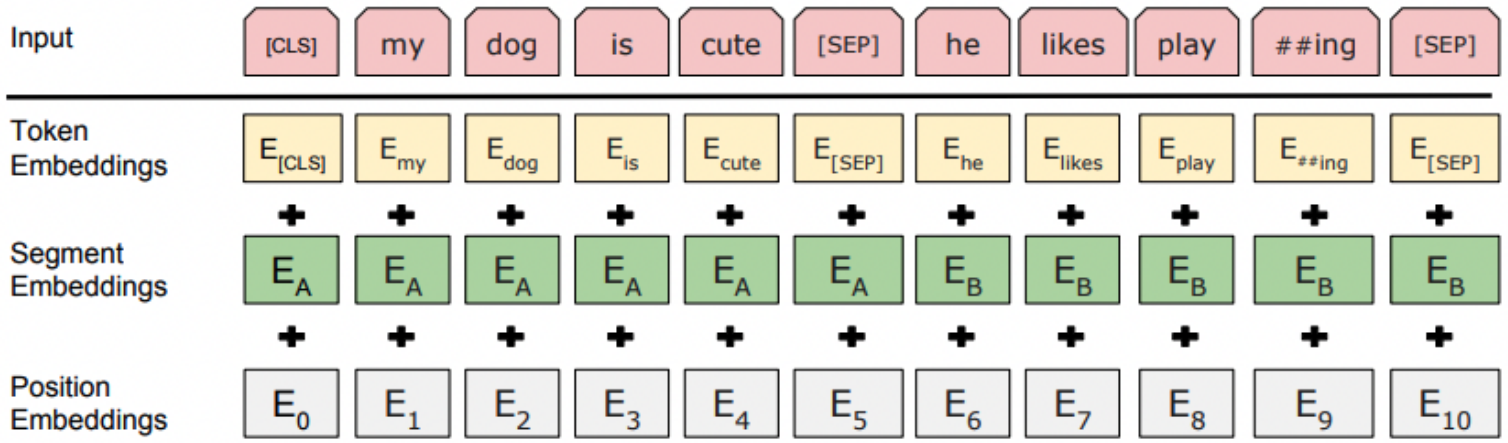
- Predict a random 15% of (sub)word tokens. If i-th token is chosen:
  - Replace this token with [MASK] 80% of the time
  - Replace this token with a random token 10% of the time
  - Leave this token unchanged 10% of the time (but still predict it!)
- Why?



[Devlin et al., 2018]

BERT: Bidirectional Encoder Representations from Transformers

- Pretraining task (2): next sentence prediction (NSP).
- Input: two separate consecutive chunks of text



- BERT was trained to predict whether one chunk follows the other or is randomly sampled.
- Later work (e.g., RoBERTa) found this "next sentence prediction" is not necessary.

[Devlin et al., 2018, Liu et al., 2019]

BERT: Bidirectional Encoder Representations from Transformers

Devlin et al., 2018 proposed a Transformer-based model called BERT.

BERT's pre-training recipe contains two tasks: Masked Language Modeling (MLM) and Next-sentence prediction (NSP).

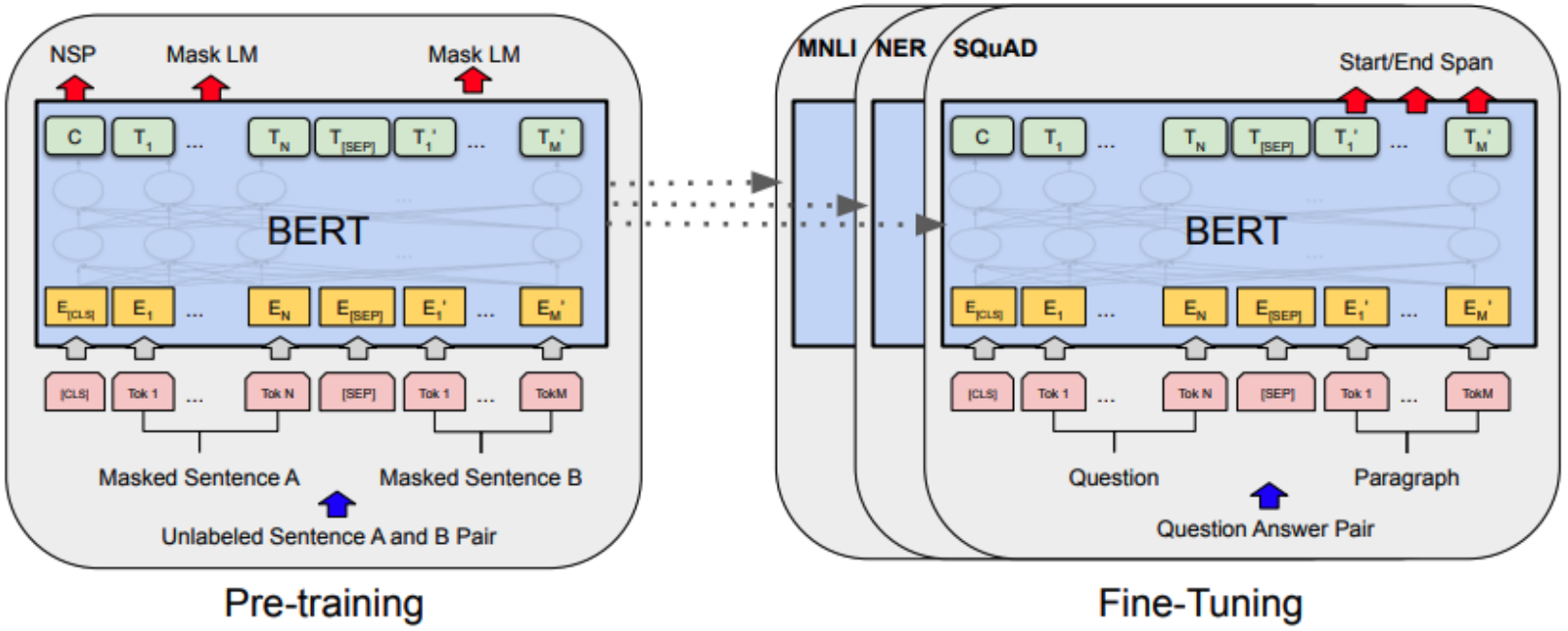


Figure 1: Overall pre-training and fine-tuning procedures for BERT. Apart from output layers, the same architectures are used in both pre-training and fine-tuning. The same pre-trained model parameters are used to initialize models for different down-stream tasks. During fine-tuning, all parameters are fine-tuned. [CLS] is a special symbol added in front of every input example, and [SEP] is a special separator token (e.g. separating questions/answers).

[Devlin et al., 2018]

BERT: Bidirectional Encoder Representations from Transformers

Other details about BERT:

- Two models were released:
  - **BERT-base**: 12 layers, 768-dim hidden states, 12 attention heads, 110 million params.
  - **BERT-large**: 24 layers, 1024-dim hidden states, 16 attention heads, 340 million params.
- Trained on:
  - BooksCorpus (800 million words)
  - English Wikipedia (2,500 million words)
- Pretraining is expensive and impractical on a single GPU.
  - BERT was pretrained with 64 TPU chips for a total of 4 days.
  - (TPUs are special tensor operation acceleration hardware)
- Finetuning is practical and common on a single GPU.
  - “Pretrain once, finetune many times.”

[Devlin et al., 2018]

BERT: Bidirectional Encoder Representations from Transformers

BERT was massively popular and hugely versatile; finetuning BERT led to new state-of-the-art results on a broad range of tasks.

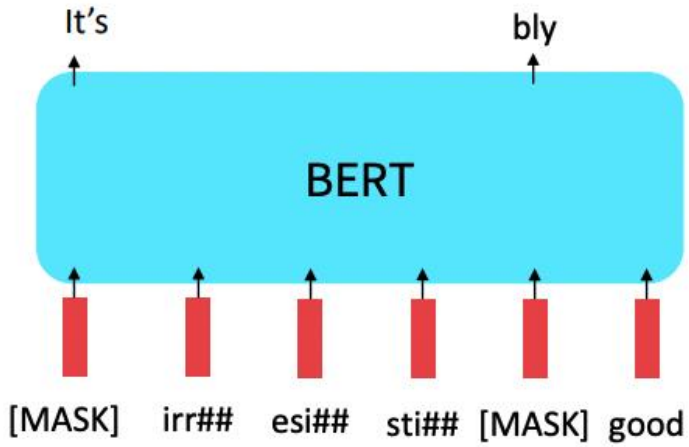
- **QQP:** Quora Question Pairs (detect paraphrase questions)
- **QNLI**: natural language inference over question answering data
- **SST-2**: sentiment analysis
- **CoLA**: corpus of linguistic acceptability (detect whether sentences are grammatical.)
- **STS-B**: semantic textual similarity
- **MRPC**: microsoft paraphrase corpus
- **RTE**: a small natural language inference corpus

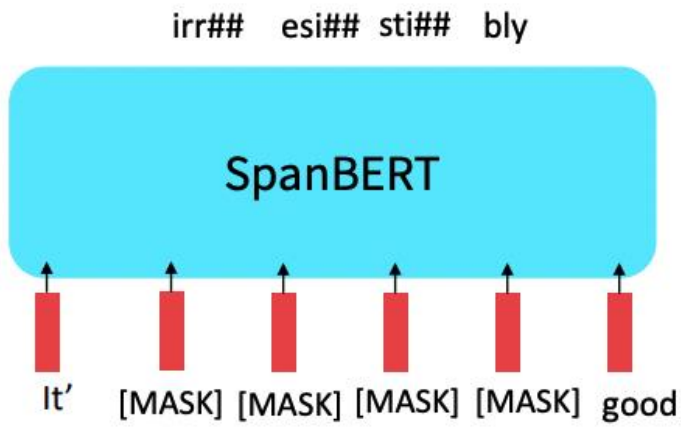
System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	**Average** -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
$\text{BERT}_{\text{BASE}}$	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
$\text{BERT}_{\text{LARGE}}$	**86.7/85.9**	**72.1**	**92.7**	**94.9**	**60.5**	**86.5**	**89.3**	**70.1**	**82.1**

[Devlin et al., 2018]

Extensions of BERT

- You'll see a lot of BERT variants like RoBERTa, SpanBERT, +++
- Some generally accepted improvements to the BERT pretraining formula:
  - RoBERTa: mainly just train BERT for longer and remove next sentence prediction!
  - SpanBERT: masking contiguous spans of words makes a harder, more useful pretraining task.





[Liu et al., 2019; Joshi et al., 2020]

Extensions of BERT

Takeaways from the RoBERTa paper:

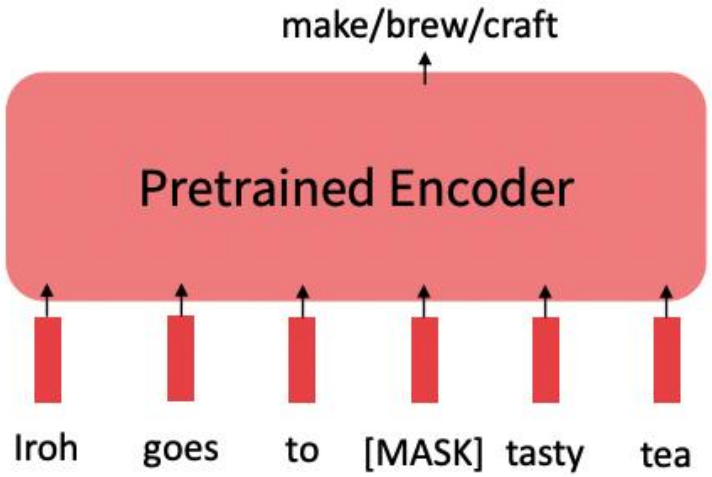
1. More compute, more data can improve pretraining even when not changing the underlying Transformer encoder.
2. Next-sentence prediction is not necessary.

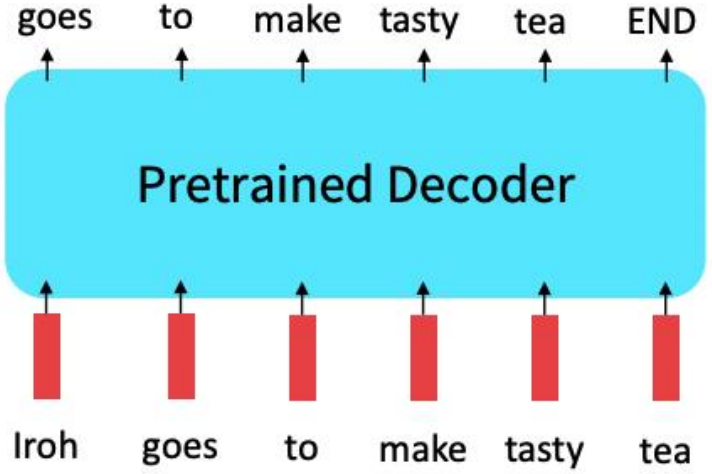
Model	data	bsz	steps	SQuAD (v1.1/2.0)	MNLI-m	SST-2
RoBERTa						
with BOOKS + WIKI	16GB	8K	100K	93.6/87.3	89.0	95.3
+ additional data (§3.2)	160GB	8K	100K	94.0/87.7	89.3	95.6
+ pretrain longer	160GB	8K	300K	94.4/88.7	90.0	96.1
+ pretrain even longer	160GB	8K	500K	**94.6/89.4**	**90.2**	**96.4**
$\text{BERT}_{\text{LARGE}}$						
with BOOKS + WIKI	13GB	256	1M	90.9/81.8	86.6	93.7

[Liu et al., 2019; Joshi et al., 2020]

Limitations of pretrained encoders

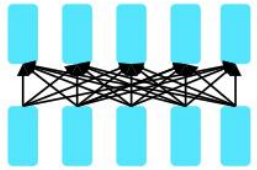
- Those results looked great! Why not used pretrained encoders for everything?
- If your task involves generating sequences, consider using a pretrained decoder.
- BERT and other pretrained encoders don't naturally lead to nice autoregressive (1-word-at-a-time) generation methods.





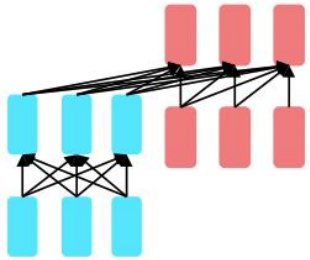
Pretraining for three types of architectures

The neural architecture influences the type of pretraining, and natural use cases.



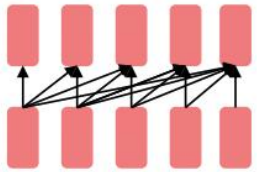
Encoders

- Gets bidirectional context – can condition on future!
- How do we train them to build strong representations?



Encoder-Decoders

- Good parts of decoders and encoders?
- What's the best way to pretrain them?



Decoders

- Language models! What we've seen so far.
- Nice to generate from; can't condition on future words

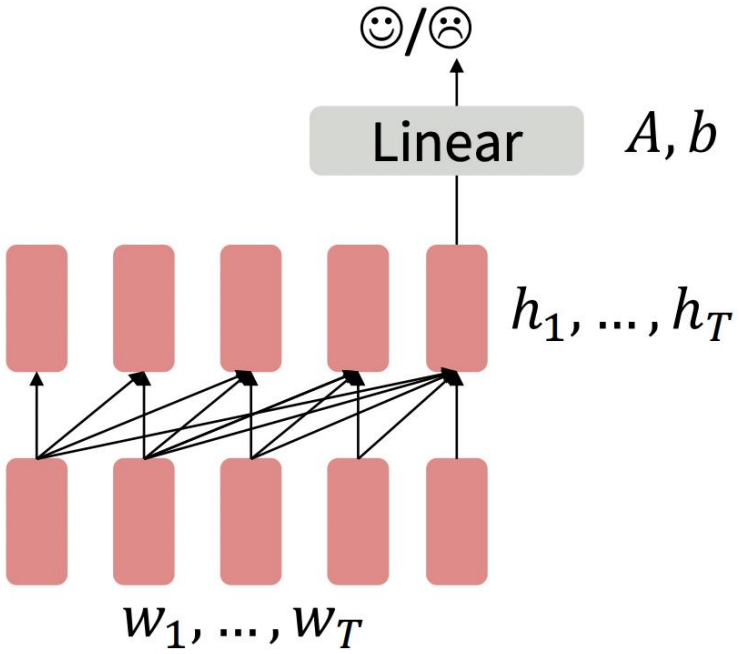
Pretraining decoders

- When using language model pretrained decoders, we can ignore that they were trained to model $p(w_t|w_{1:t-1})$

- We can finetune them by training a classifier on the last word's hidden state.

$$h_1, \ldots, h_T = \text{Decoder}(w_1, \ldots, w_T)$$
$$y \sim Ah_T + b$$

- Where A and b are randomly initialized and specified by the downstream task.
- Gradients backpropagate through the whole network.



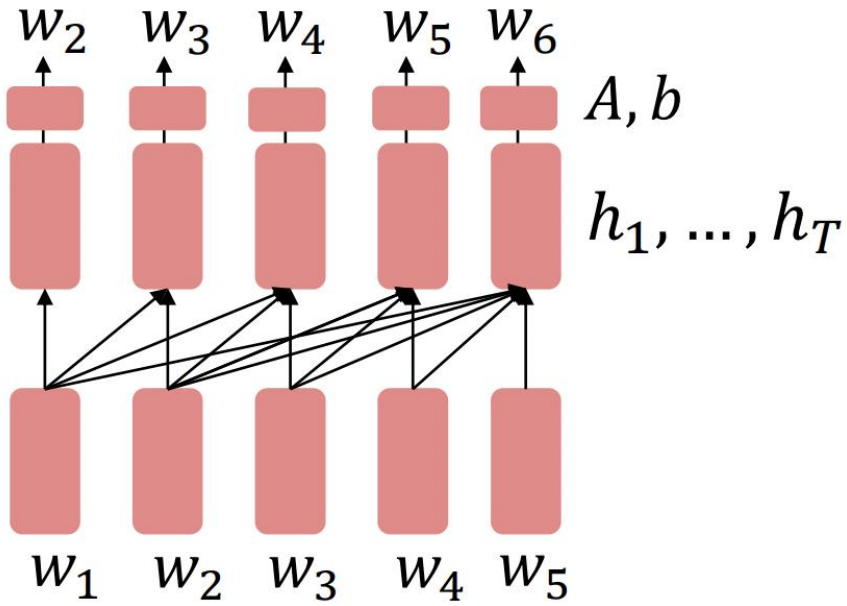
[Note how the linear layer hasn't been pretrained and must be learned from scratch.]

Pretraining decoders

- It's natural to pretrain decoders as language models and then use them as generators, finetuning their $p(w_t|w_{1:t-1})$
- This is helpful in tasks where the output is a sequence with a vocabulary like that at pretraining time!
  - Dialogue (context=dialogue history)
  - Summarization (context=document)

$$h_1, \dots, h_T = \text{Decoder}(w_1, \dots, w_T)$$
$$w_t \sim Ah_{t-1} + b$$

- Where A, b were pretrained in the language model!



Generative Pretrained Transformer (GPT)

2018's GPT was a big success in pretraining a decoder!

- Transformer decoder with 12 layers, 117M parameters.
- 768-dimensional hidden states, 3072-dimensional feed-forward hidden layers.
- Byte-pair encoding with 40,000 merges
- Trained on BooksCorpus: over 7000 unique books.
  - Contains long spans of contiguous text, for learning long-distance dependencies.
- The acronym "GPT" never showed up in the original paper; it could stand for "Generative PreTraining" or "Generative Pretrained Transformer"

[Radford et al., 2018]

Generative Pretrained Transformer (GPT)

GPT (fine-tuned on downstream tasks) results on various natural language inference datasets.

Method	MNLI-m	MNLI-mm	SNLI	SciTail	QNLI	RTE
ESIM + ELMo [44] (5x)	-	-	89.3	-	-	-
CAFE [58] (5x)	80.2	79.0	89.3	-	-	-
Stochastic Answer Network [35] (3x)	80.6	80.1	-	-	-	-
CAFE [58]	78.7	77.9	88.5	83.3		
GenSen [64]	71.4	71.3	-	-	82.3	59.2
Multi-task BiLSTM + Attn [64]	72.2	72.1	-	-	82.1	**61.7**
Finetuned Transformer LM (ours)	**82.1**	**81.4**	**89.9**	**88.3**	**88.1**	56.0

[Radford et al., 2018]

Increasingly convincing generations (GPT2)

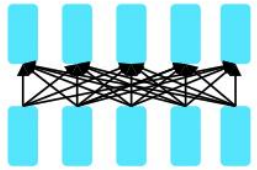
- We mentioned how pretrained decoders can be used in their capacities as language models. GPT-2, a larger version (1.5B) of GPT trained on more data, was shown to produce relatively convincing samples of natural language.

Context (human-written): In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.
GPT-2: The scientist named the population, after their distinctive horn, Ovid's Unicorn. These four-horned, silver-white unicorns were previously unknown to science. Now, after almost two centuries, the mystery of what sparked this odd phenomenon is finally solved. Dr. Jorge Pérez, an evolutionary biologist from the University of La Paz, and several companions, were exploring the Andes Mountains when they found a small valley, with no other animals or humans. Pérez noticed that the valley had what appeared to be a natural fountain, surrounded by two peaks of rock and silver snow.

[Radford et al., 2018]

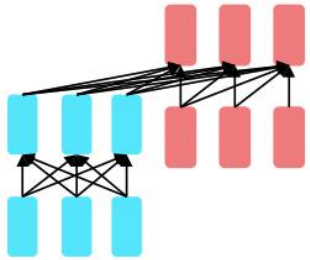
Pretraining for three types of architectures

The neural architecture influences the type of pretraining, and natural use cases.



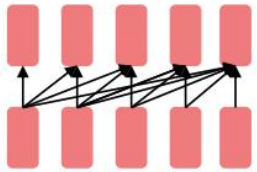
Encoders

- Gets bidirectional context – can condition on future!
- How do we train them to build strong representations?



Encoder-Decoders

- Good parts of decoders and encoders?
- What's the best way to pretrain them?



Decoders

- Language models! What we've seen so far.
- Nice to generate from; can't condition on future words

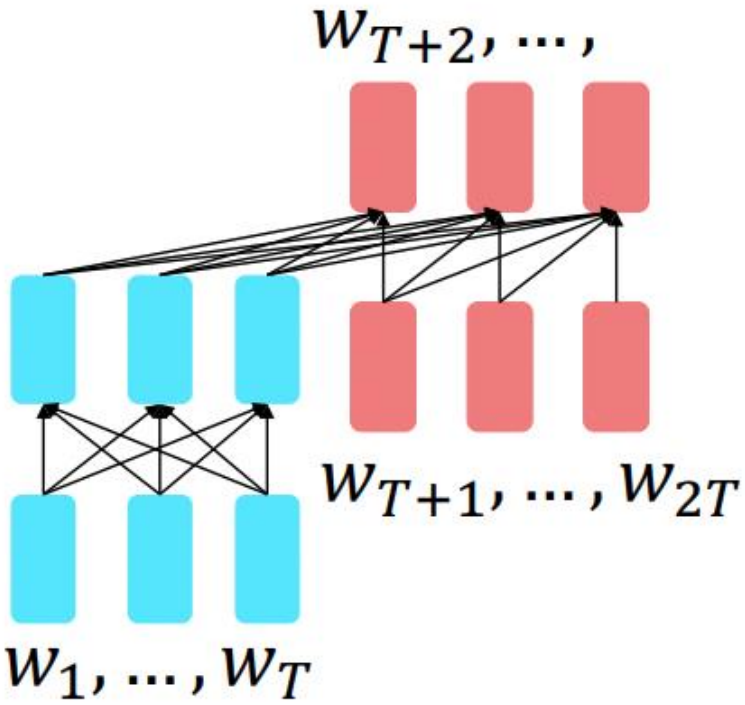
Pretraining encoder-decoders: what pretraining objective to use?

- Encoder-decoders are pre-trained with objectives resembling language modeling
- But a prefix, $w_1, \dots, w_T$ is provided to the encoder and is not predicted.

$$h_1, \dots, h_T = \text{Encoder}(w_1, \dots, w_T)$$
$$h_{T+1}, \dots, h_{2T} = Decoder(w_1, \dots, w_T, h_1, \dots, h_T)$$
$$y_i \sim Ah_i + b, i > T$$

- The encoder portion benefits from bidirectional context.

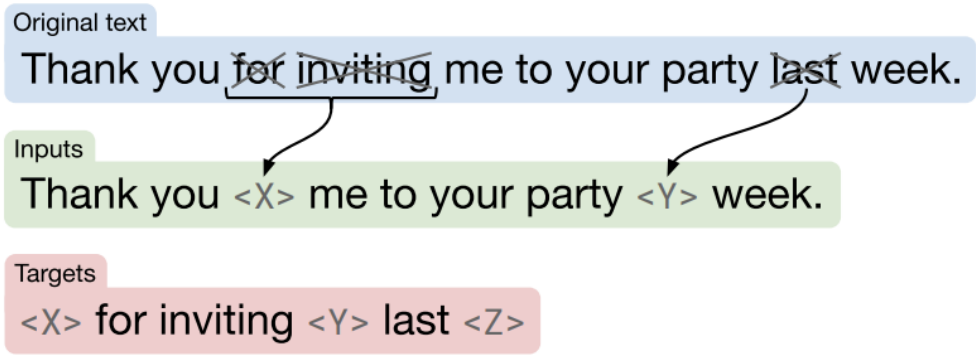
- The decoder portion is used to train the whole model through language modeling.



[Raffel et al., 2018]

Pretraining encoder-decoders: what pretraining objective to use?

- Text-to-Text Transfer Transformer (T5) [Raffel et al., 2018]
- Replace different-length spans from the input with unique placeholders; decode out the spans that were removed!



Trends in modern LM pretraining

Models are becoming larger and larger

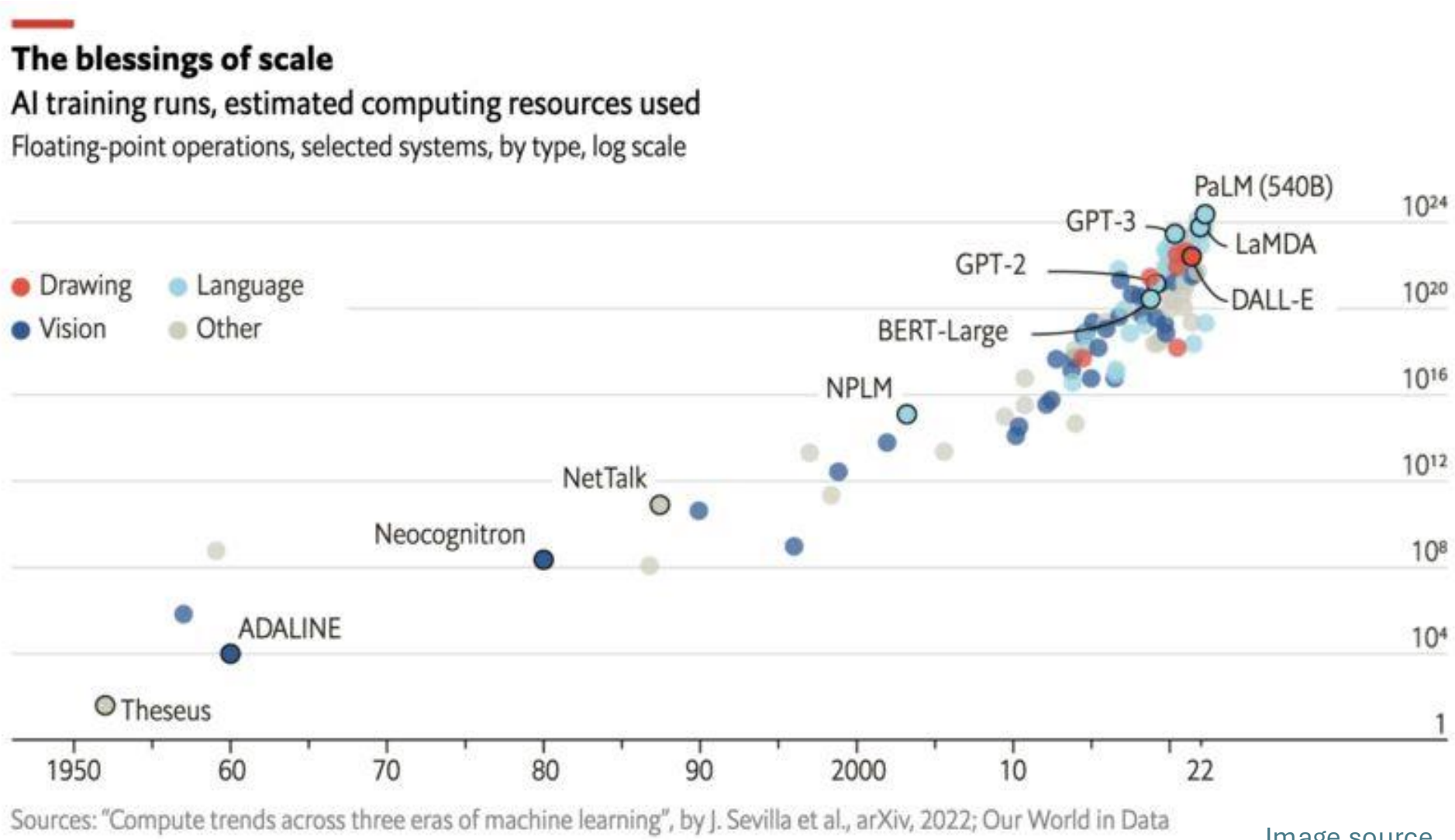
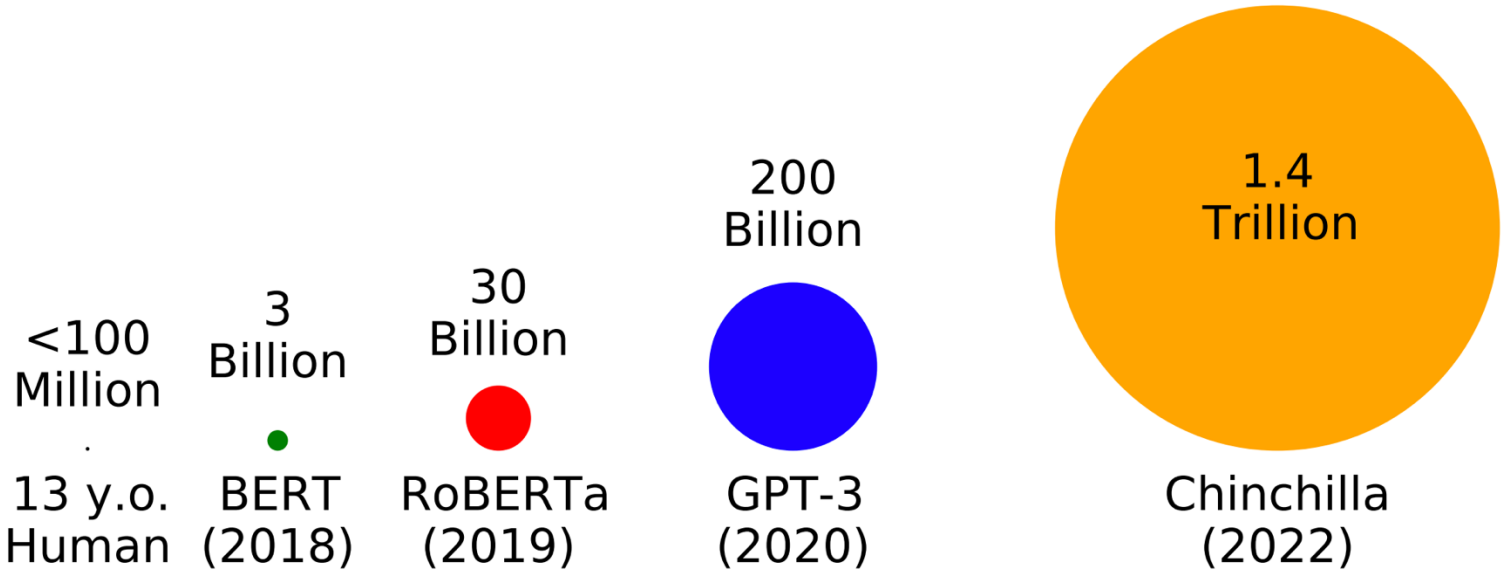


Image source

Trained on more and more data

tokens seen during training

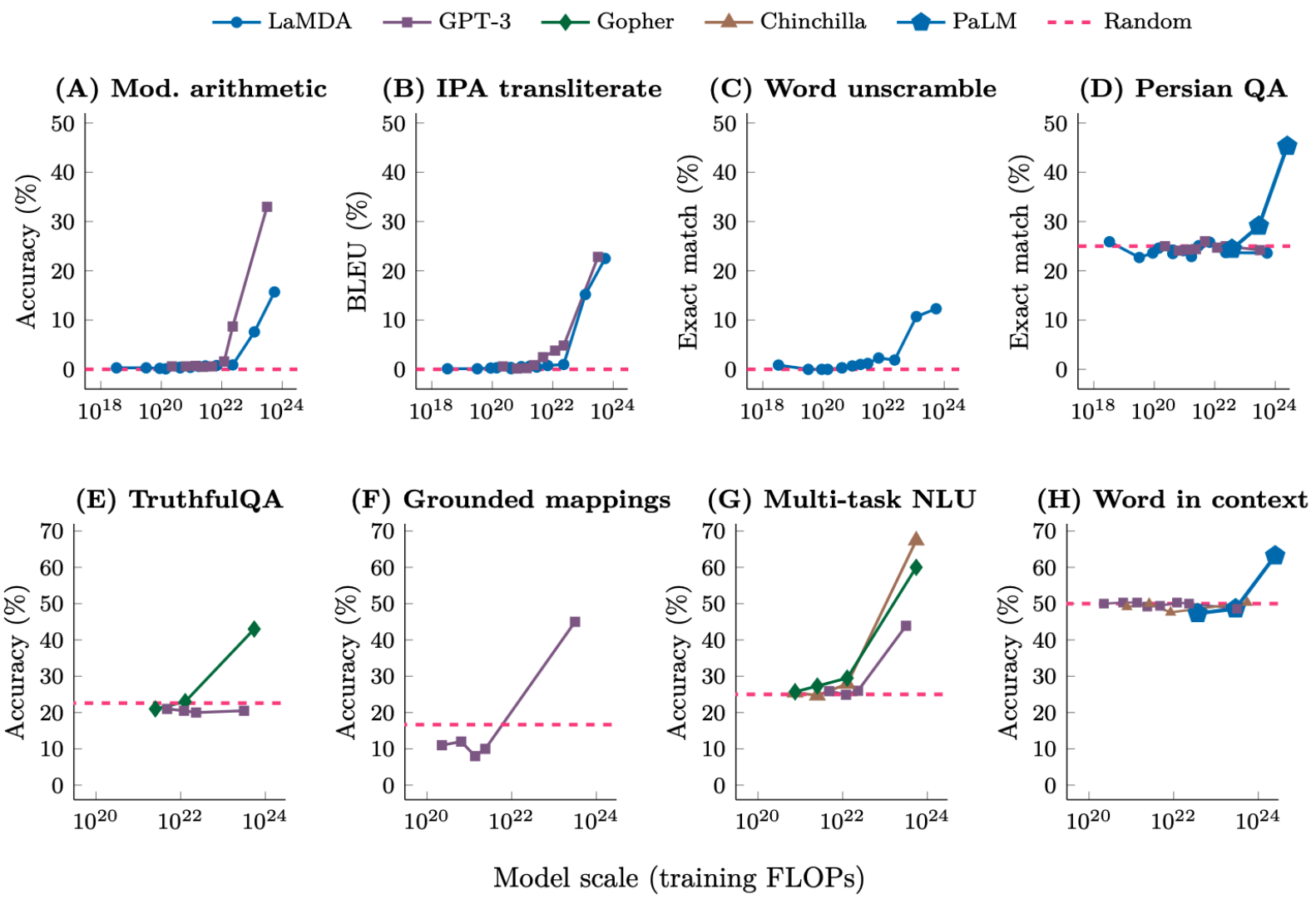
- Huge effort has been put into optimizing LM pretraining at massive scales in the last several years.
- Datasets have also grown by orders of magnitude.



https://babylm.github.io/

Why larger models?

- Wei et al (2022) summarized that the larger models show emerging capabilities.



Scaling up the model and data is rewarding

- Hoffman et al (2020) summarized a scaling law: as the models become larger and the training data becomes larger, the loss can be predictively better.

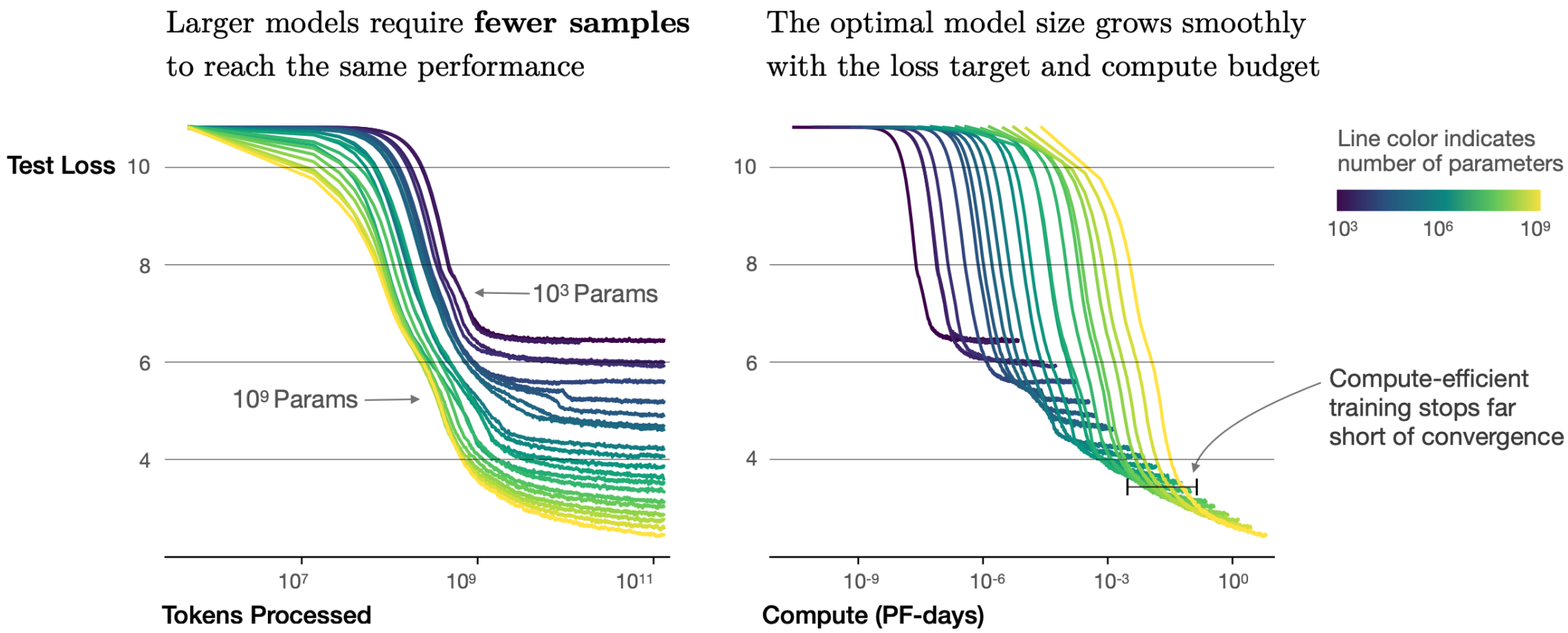


Figure 2 We show a series of language model training runs, with models ranging in size from 10^3 to 10^9 parameters (excluding embeddings).

The three factors of compute scaling

- Researchers found that empirically, the LM performance can be predicted by three factors:
  - Model size (in terms of N. parameters) N
  - Dataset size (in terms of tokens) D
  - Compute (in terms of FLOP) C
- Given a fixed computation budget C, Hoffman et al (2022) estimated that the optimal model size and dataset sizes are:

$$N_{opt} = G \times \left(\frac{C}{6}\right)^{a}$$

$$D_{opt} = G^{-1} \times \left(\frac{C}{6}\right)^{b}$$

where G is a constant, $\mathrm{a} = 0.46$, and $b = 0.54$.

This means that the optimal N and the optimal D should scale similarly with C.

Part 2

Post-training / Fine-tuning

Post-training

- After pre-training, LLMs can acquire the general abilities for solving various tasks.
- However, an increasing number of studies have shown that LM's abilities can be further adapted according to specific goals.
- Let's start with several examples of post-training tasks of BERT model (fine-tuning)

BERT's fine-tuning

- When fine-tuning, attach an additional Linear classification head.
- This head is trained from scratch.
- Depending on the tasks, the representations of different tokens are passed into the classification head.

[Devlin et al., 2018]

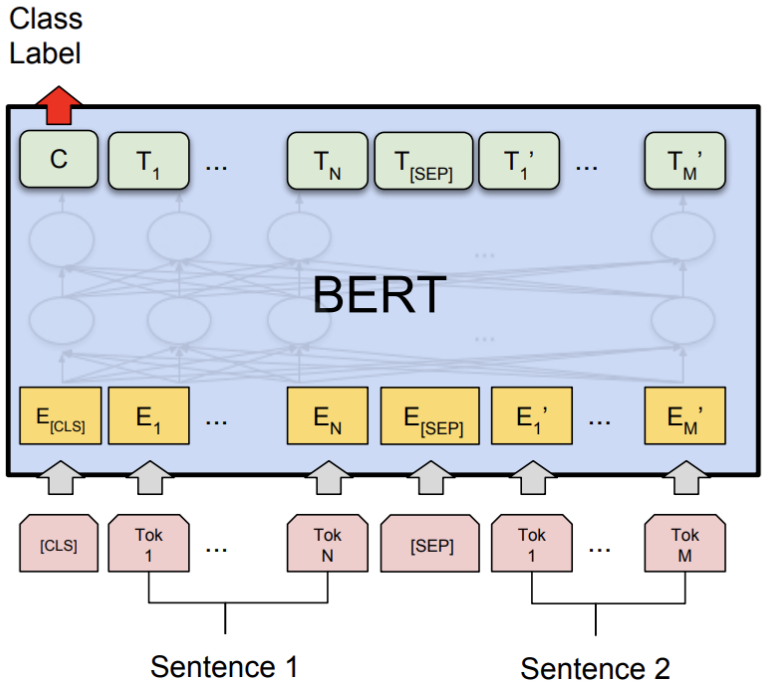
BERT's fine-tuning

MNLI is an example of a sentence-pair classification task.
Following are two examples:

Premise: "The cat sat on the mat."
Hypothesis: "The cat did not sit on the mat."
Label: "Contradictory"

Premise: "And it's nice talking with you alrighty"
Hypothesis: "I talk to you everyday."
Label: 'Neutral"

Template: [CLS] + Premise + [SEP] + Hypothesis
Use the representation of the [CLS] token.



(a) Sentence Pair Classification Tasks: MNLI, QQP, QNLI, STS-B, MRPC, RTE, SWAG

[Devlin et al., 2018]

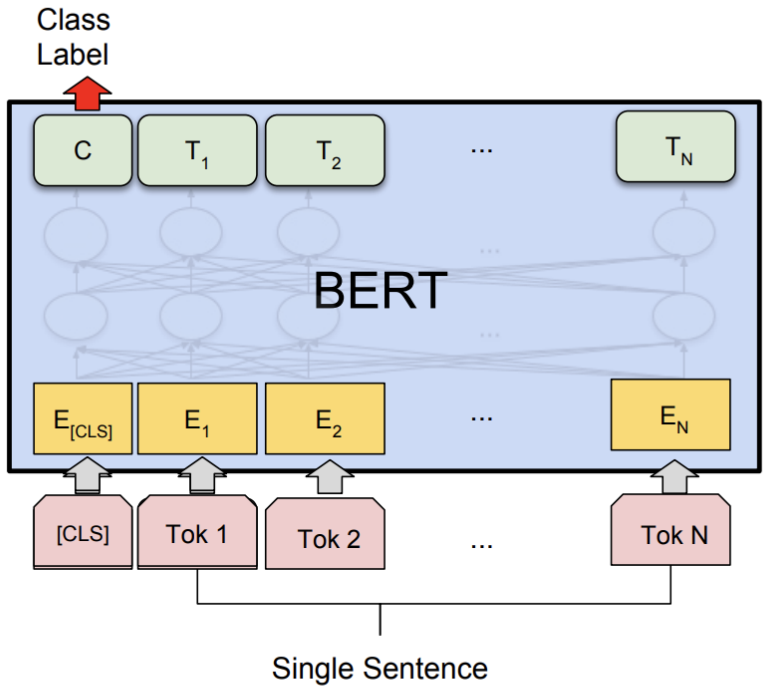
BERT's fine-tuning

SST-2 is an example of single-sequence classification tasks.
Following are two examples:

Sentence: "Goes to absurd lengths"
Label: Negative

Sentence: "The greatest musicians"
Label: Positive

Template: [CLS] + Sentence
Use the representation of the [CLS] token.



(b) Single Sentence Classification Tasks: SST-2, CoLA

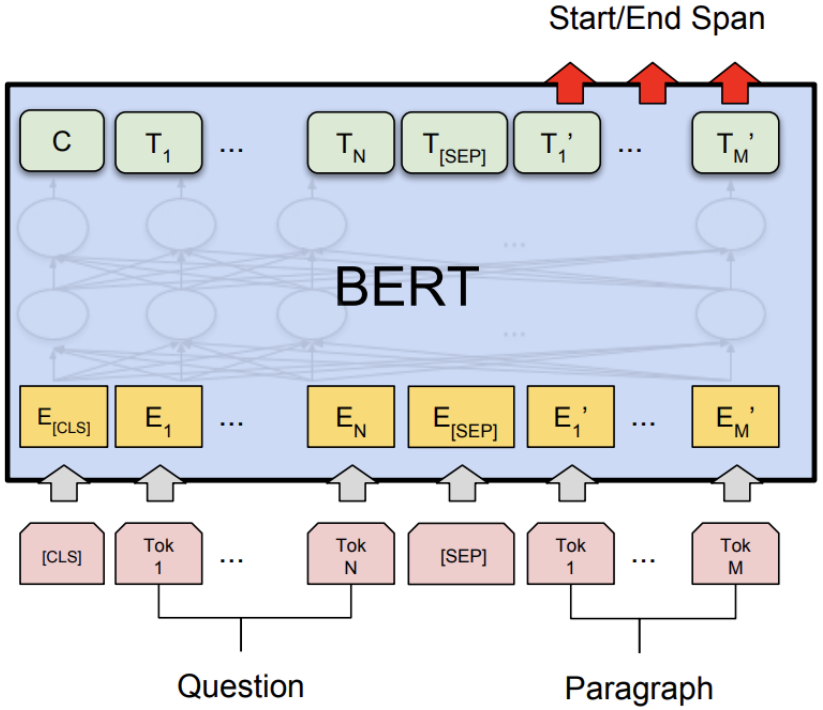
[Devlin et al., 2018]

BERT's fine-tuning

SQuAD is an example of Question Answering tasks. Following is an example:

Context: [...] Notre Dame's students run a number of news media outlets. [...] Begun as a one-page journal in September 1876, the Scholastic magazine is issued twice monthly and claims to be the oldest continuous collegiate publication in the United States. [...]
Question: When did the Scholastic Magazine of Notre dame begin publishing?
Answer: September 1876.

Template: [CLS] + Context + [SEP] + Answer
Use the representations of all the answer tokens.



(c) Question Answering Tasks: SQuAD v1.1

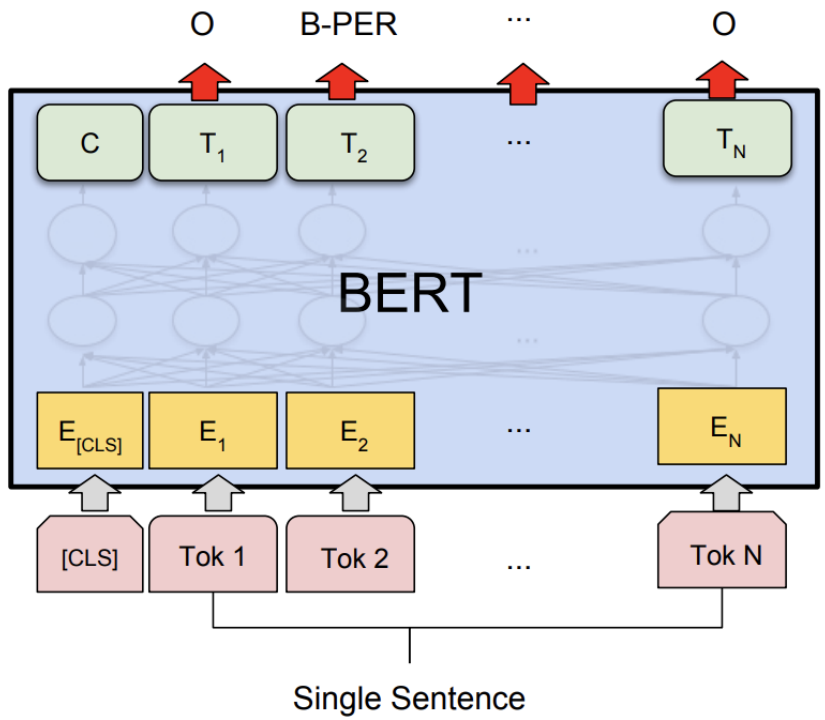
[Devlin et al., 2018]

BERT's fine-tuning

CoNLL-2003 NER is an example of the span-level tagging tasks. Following is an example:

Text: Looking for Ring. Do you sell it ?
Label: 0 0 1 0 0 0 0 0

Template: [CLS] + Text
Use the representations of all tokens. Each token has a label.



(d) Single Sentence Tagging Tasks: CoNLL-2003 NER

[Devlin et al., 2018]

Supervised Fine-Tuning / Instruction Tuning

- Recall the “teacher-forcing” setting in Seq2Seq models?
- SFT uses the same setting to fine-tune the language model $p(.)$.

- Idea: A sequence $[w_1, w_2, \dots, w_k, w_{k+1}, \dots, w_n]$ is sampled from the dataset, where:
  - $[w_1, w_2, \dots, w_k]$ is the instruction
  - $[w_{k+1}, \dots, w_n]$ is the human-written response

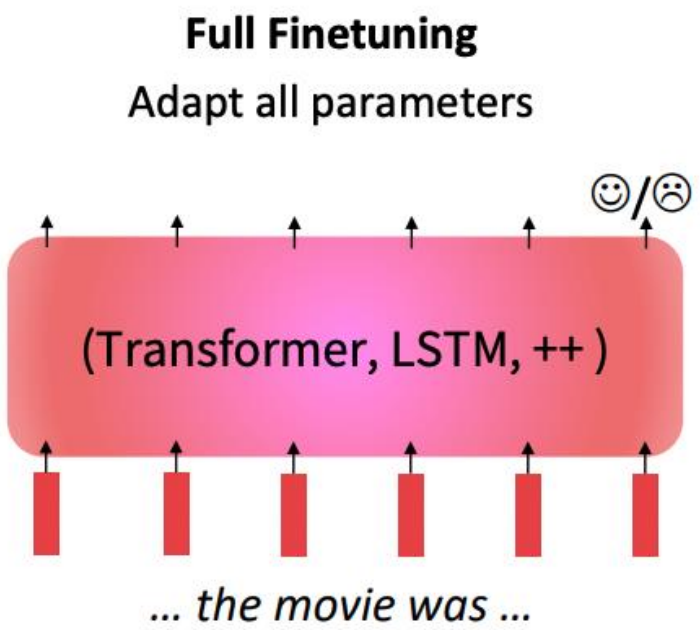
- SFT minimizes the cross-entropy loss of the model predicting the response tokens, given the instruction and the prior ground-truth response tokens:

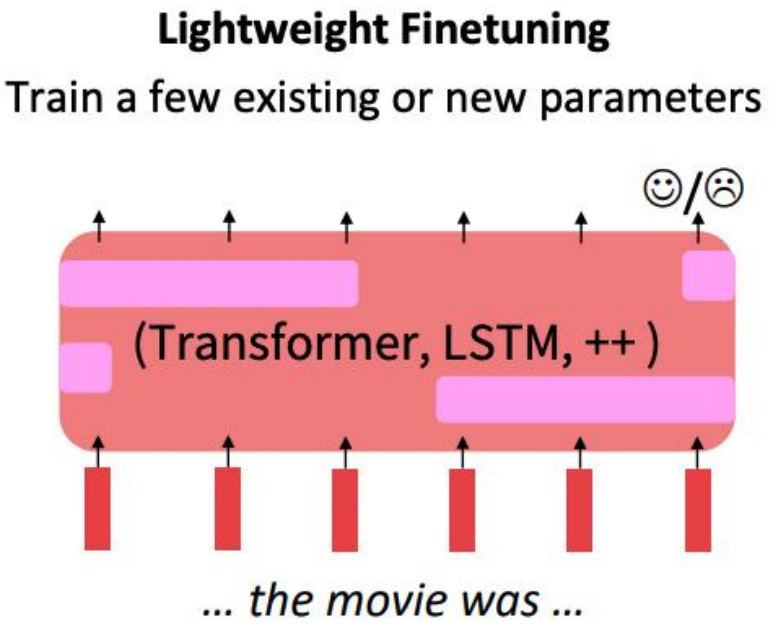
$$J = -\frac{1}{n-k}\sum_{i=k+1}^{n} y_i \log p\,(w_i | w_1 .. w_n, w_{k+1} .. w_{i-1})$$

Parameter-Efficient Finetuning

Full Finetuning vs. Parameter-Efficient Finetuning

- Full finetuning: Finetuning every parameter in a pretrained model works well, but is memory-intensive.
- Parameter-efficient finetuning: But lightweight finetuning methods adapt pretrained models in a constrained way. Leads to less overfitting and/or more efficient finetuning and inference.

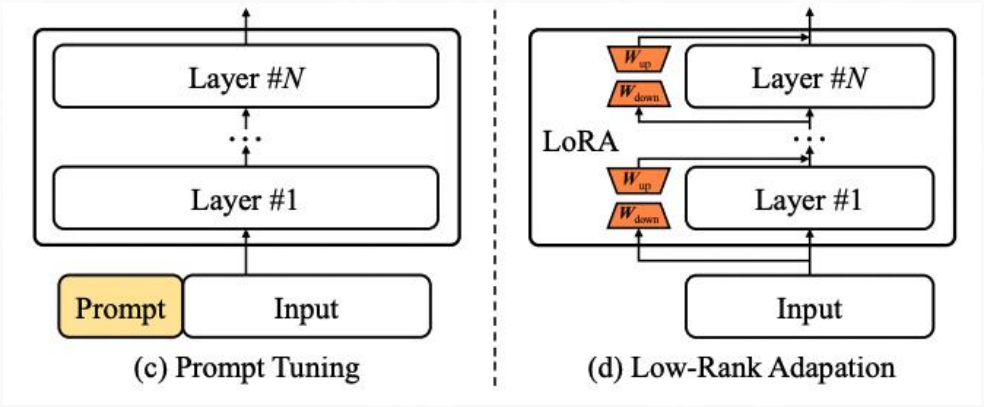


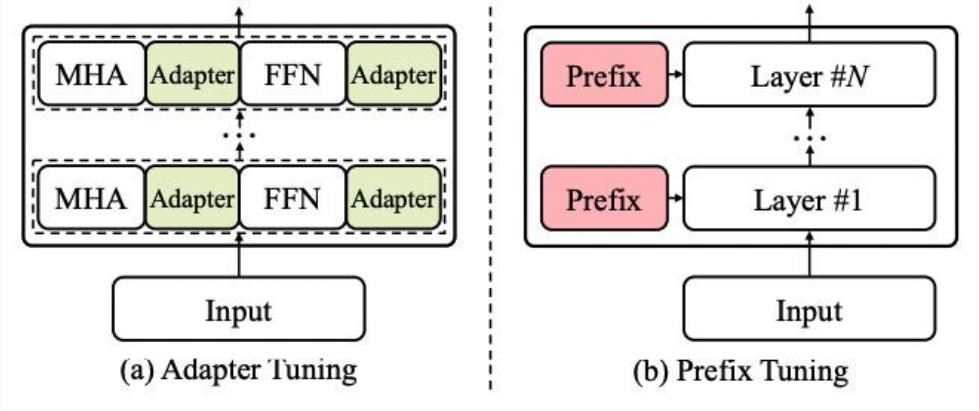


[Liu et al., 2019; Joshi et al., 2020]

Parameter-Efficient Fine-Tuning Methods

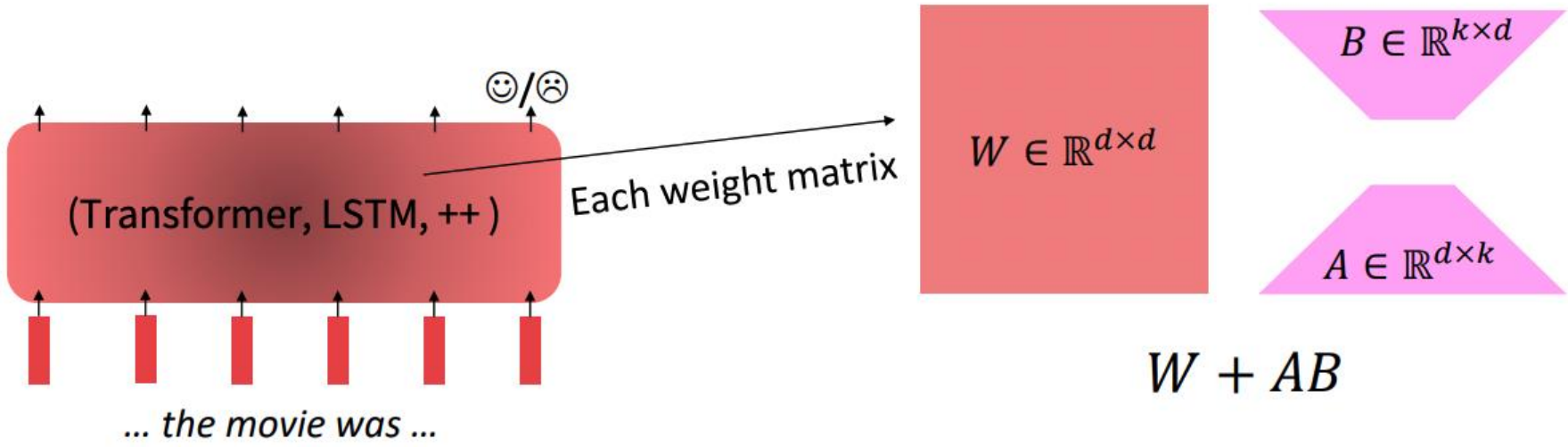
Aims to reduce the number of trainable parameters while retaining a good performance





Low-Rank Adaptation (LoRA)

- Low-Rank Adaptation learns a low-rank "diff" between the pretrained and finetuned weight matrices.



A and B are trainable parameters for task adaption. k is the reduced rank.

How is LoRA efficient? Because $k \ll d$, so LoRA only has $2kd$ trainable parameters (A and B), compared to FT's d^2 parameters, per W.

[Hu et al., 2021]

Parameter-Efficient Finetuning

Hugging Face PEFT: https://github.com/huggingface/peft/tree/main

🤗 PEFT

State-of-the-art Parameter-Efficient Fine-Tuning (PEFT) methods

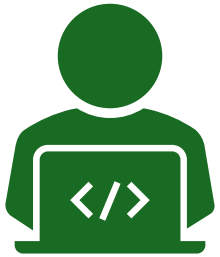
Parameter-Efficient Fine-Tuning (PEFT) methods enable efficient adaptation of pre-trained language models (PLMs) to various downstream applications without fine-tuning all the model's parameters. Fine-tuning large-scale PLMs is often prohibitively costly. In this regard, PEFT methods only fine-tune a small number of (extra) model parameters, thereby greatly decreasing the computational and storage costs. Recent State-of-the-Art PEFT techniques achieve performance comparable to that of full fine-tuning.

Seamlessly integrated with 🤗 Accelerate for large scale models leveraging DeepSpeed and Big Model Inference.

Supported methods:

1. LoRA: LORA: LOW-RANK ADAPTATION OF LARGE LANGUAGE MODELS
2. Prefix Tuning: Prefix-Tuning: Optimizing Continuous Prompts for Generation, P-Tuning v2: Prompt Tuning Can Be Comparable to Fine-tuning Universally Across Scales and Tasks
3. P-Tuning: GPT Understands, Too
4. Prompt Tuning: The Power of Scale for Parameter-Efficient Prompt Tuning
5. AdaLoRA: Adaptive Budget Allocation for Parameter-Efficient Fine-Tuning
6. $(IA)^3$: Few-Shot Parameter-Efficient Fine-Tuning is Better and Cheaper than In-Context Learning
7. MultiTask Prompt Tuning: Multitask Prompt Tuning Enables Parameter-Efficient Transfer Learning
8. LoHa: FedPara: Low-Rank Hadamard Product for Communication-Efficient Federated Learning

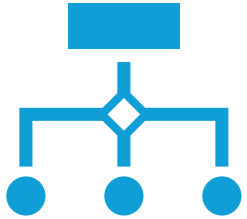
Summary



Pre-training

Algorithms and architectures

Data, and the scaling law



Post-training

Supervised finetuning

Parameter-efficient finetuning

Readings

1. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding
2. Contextual Word Representations: A Contextual Introduction
3. The Illustrated BERT, ELMo, and co.
4. Martin & Jurafsky Chapter on Transfer Learning

Slides prepared based on

- A Survey of Large Language Models
- CS224: Natural language processing with deep learning at Stanford
- CS388: Natural Language Processing at UT Austin
- Instructor slides from Dr. Yue Ning, Dr. Ping Wang, and Dr. Zining Zhu